# Multi-environment CI/CD DevOps Infrastructure for a Full-stack TypeScript Web App

John Swärd - johnsw@kth.se
Dimitrios Bakalis - bakalis@kth.se

Our application is a WebApp written in TypeScript. It is a quiz game called "Absurdly True?"

The application is split in three services:

Frontend: a single page application built with React and Vite. The built static files are served by an nginx web server.
Backend: a REST API built with Node.js and Express.
Database: a PostgreSQL database that stores the scores. The database schema is managed with SQL migrations, which run automatically when the backend starts.

Technologies used for the application:

- TypeScript, for both frontend and backend
- React 19 and Vite
- nginx (serving the frontend and acting as a reverse proxy)
- Node.js 22 and Express 5
- PostgreSQL 17
- Vitest (unit tests) and Supertest (API tests)
- Oxlint (linting and formatting)
- npm
- Docker

## Continuous Integration

CI runs on GitHub Actions and runs for every pull request into dev, test, or main. Branch protection requires it to pass before it can be merged, so nothing reaches a deployable branch without going through it. The workflow is defined in .github/workflows/ci.yml.

We are using:

- **Path filtering**. The first job checks which part of the repository the PR has touched and only runs checks belonging to that directory. For example, backend checks will only run on backend/** changes, frontend on frontend/** changes, and a change to the workflow file will run everything. This keeps the builds running quickly for small changes.
- **A single check**. Jobs skipped by path filtering never report a status, so they can't be required checks. A final ci-ok job waits for all other jobs and only fails if one of them fails. This is the only required check, so skipped jobs count as passing while real failures still block the merge.

- **Build and test**. The frontend and backend both run the same steps. We install dependencies with npm ci, lint with Oxlint where warnings count as errors, type check with tsc, run the tests with Vitest, and run a production build. Front-end tests cover the game logic and back-end tests cover the request validation and REST API, using Supertest.
- **Database integration tests**. The backend job starts a PostgreSQL container, so the tests for the database layer run against a real database instead of being skipped.
- **Migration compatibility**. When a pull request changes the database migrations, a separate job checks two things. First, it blocks changes to migrations that have already been released, so new changes have to go in a new migration file. Second, it applies the new migrations and runs the previous version's backend tests against the updated database. This makes sure the old backend keeps working during a deploy, while it still runs against the new database.

## Security

Security checks run as part of the same CI pipeline, so a security finding blocks the merge just like a failing test.

We use:

- **Gitleaks** for scanning the commits in each pull request for leaked passwords, keys, and tokens before they are merged into dev, test or main.
- **Dependency review** will fail a pull request that adds a dependency with a known vulnerability of high severity or above.
- **Trivy** for scanning the frontend and backend Docker images. The build fails on high or critical vulnerabilities that have a fix available.
- **Dependabot** for opening weekly pull requests to update npm packages, Docker base images and GitHub Actions. Minor and patch updates are grouped into one pull request to reduce noise. All of these go through the same CI as any other change.

## Continuous Deployment

As part of this project, we also created some Continuous Deployment pipelines.
The features that we provide are the following:

- Three environments (dev, test, prod), connected to three branches in our repository (dev, test, main respectively).
- Every push to one of these branches triggers a Deployment on the respective environment.
- Deployment can also be run manually.
- You cannot push directly to the main branch, but only through a pull request.
- Deployments run on the same machine for the scope of this project, but they run on isolated docker networks.

- Deployments are orchestrated using Infrastructure as Code
- Infrastructure as Code state is stored on a MinIO instance.
- Automatic health checks on every deployment.
- Minimal integration tests when deploying to the test environment.
- Only the frontend publishes a port on the host machine.
- Support of multiple architectures (amd64 / arm64) on built Docker Images.
- Automatic Github Releases on Pull Requests merged to main (semantic versioning through Github labels).
- Github secrets used for sensitive information.
- Teardown Github action to manually take down environments

To build these features we use the following technologies:

The Github platform, especially:
- Github repositories
- Github actions
- Github secrets
- Github runners to run actions locally on-prem
- Github environments
- Github branch protection rules
- Github Container Registry
- Docker
- Terraform
- MinIO Object Store

## Infrastructure as Code with Terraform

We have one single Terraform configuration that is shared by all three environments. It describes the three containers of the application (database, backend and frontend), and a Docker network and a volume for the database data. Terraform waits for a container to be healthy before creating the ones that depend on it.
The environments differ only in a small .tfvars file, which contains the environment name and the port of the frontend (8081 for dev, 8082 for test and 8080 for prod). All the other values, like the container ports, the database name and user, and the health check settings, are variables with default values, and can be changed in the same file. Because of this, adding a new environment only needs one new file, instead of copying a whole directory that can drift apart over time.
The Terraform state is stored in a MinIO instance, which works like an S3 bucket. Each environment has its own state file. The state is locked while Terraform is running, which stops two deployments from changing the same environment at the same time. The MinIO instance is automatically started by the first deployment.
In the pipeline, Terraform is run by the deployment workflow on the self-hosted runner (init, plan and apply).

## Limitations of our implementation:

Currently, all environments use the same Github runner and are deployed on the same machine. This should not be the case in a real-life deployment.
We are also using a local Github runner, which has root access to the local machine. This means that whoever is using a likewise solution should take extra care of who has the privileges to commit in the deployed branches.
No rollback strategy implemented. Currently, deployments are only driven through pushes to branches. We did this because this would add complexity to the solution and would require a more sophisticated database migration strategy.
No zero downtime implemented. Currently, no blue / green deployment, because no rollback strategy was implemented, which typically is required for blue / green deployment.

## Relevance for DevOps:

In many software companies, managing multiple environments is a pain point, especially when deployments are done manually. This project shows that DevOps practices can significantly help with this. They enforce a structure on the environments: dev, test and prod are defined from the same code and only differ in a few declared values, so they cannot slowly drift apart. The management of the environments is also code driven. Creating, updating and removing an environment is done through versioned configuration and an automated pipeline, without needing someone to take care of it manually. Every change goes through the same path, which makes deployments repeatable and easy to trace.